

WEB APPLICATION DEVELOPMENT

LAB 7 - HOMEWORK

EXERCISE 5: ADVANCED SEARCH

5.1: Multi-Criteria Search

Add to ProductRepository:

```
@Query("SELECT p FROM Product p WHERE " +
        "(:name IS NULL OR p.name LIKE %:name%) AND " +
        "(:category IS NULL OR p.category = :category) AND " +
        "(:minPrice IS NULL OR p.price >= :minPrice) AND " +
        "(:maxPrice IS NULL OR p.price <= :maxPrice)")
List<Product> searchProducts(
    @Param("name") String name,
    @Param("category") String category,
    @Param("minPrice") BigDecimal minPrice,
    @Param("maxPrice") BigDecimal maxPrice
);
```

Add to Service interface and implementation.

```
List<Product> advancedSearch(String name, String category, BigDecimal minPrice, BigDecimal maxPrice);

@Override
public List<Product> advancedSearch(String name, String category, BigDecimal minPrice, BigDecimal maxPrice) {
    return productRepository.searchProducts(name, category, minPrice, maxPrice);
}
```

Add to Controller:

```
// Advanced search products
@GetMapping("/advanced-search")
public String advancedSearch(
    @RequestParam(required = false) String name,
    @RequestParam(required = false) String category,
    @RequestParam(required = false) BigDecimal minPrice,
    @RequestParam(required = false) BigDecimal maxPrice,
    Model model) {

    List<Product> results = productService.advancedSearch(name, category, minPrice, maxPrice);

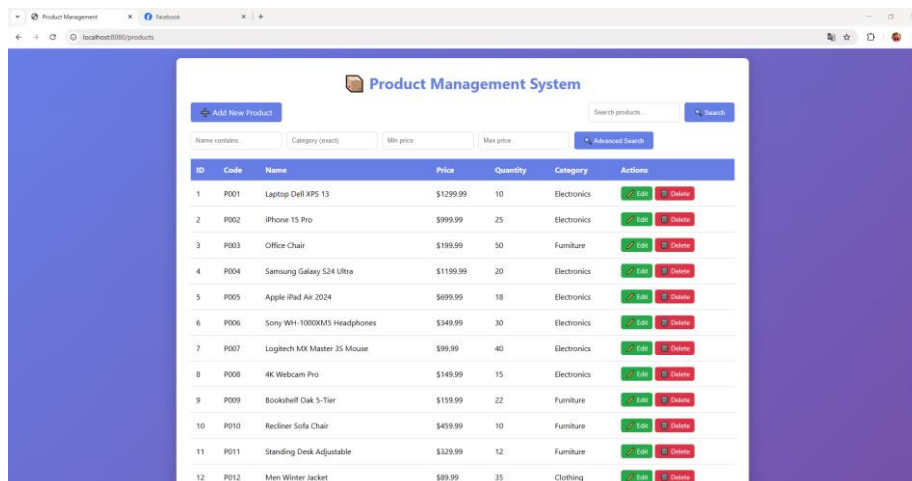
    model.addAttribute("products", results);
    model.addAttribute("name", name);
    model.addAttribute("category", category);
    model.addAttribute("minPrice", minPrice);
    model.addAttribute("maxPrice", maxPrice);

    return "product-list";
}
```

Add advanced search form to product-list.html.

```
<form th:action="@{/products/advanced-search}" method="get" class="search-form" style="margin-top:10px;">
    <input type="text" name="name" th:value="${name}" placeholder="Name contains...">
    <input type="text" name="category" th:value="${category}" placeholder="Category (exact)" />
    <input type="number" step="0.01" name="minPrice" th:value="${minPrice}" placeholder="Min price">
    <input type="number" step="0.01" name="maxPrice" th:value="${maxPrice}" placeholder="Max price">

    <button type="submit" class="btn btn-primary">🔍 Advanced Search</button>
</form>
```



Explanation about the code flow of advanced search:

1. Browser → GET /products/advanced-search
 - After user fills in the advanced search including name, category, min price and max price, they click the advanced search button.
 - The browser sends a request to ProductController such as: GET /products/advanced-search?name=iPhone&category=Electronics&minPrice=500&maxPrice=1500
2. ProductController receives the request (@GetMapping).
 - The controller extracts the query parameters such as name, category, minPrice, maxPrice.
3. Controller calls productService.advancedSearch(...)
 - Controller does not know how to search. Therefore, the service layer (ProductService.java and ProductServiceImpl.java) handles it.
4. Service calls productRepository.searchProducts(...)
5. Spring Data JPA generates dynamic JPQL query. JPA converts the custom query in ProductRepository.java file to SQL based on provided parameters. If a field is empty, JPA ignores it due to OR :param IS NULL.
6. Repository returns list of matching products.
 - JPA maps each row to Product entity.
 - This List<Product> is returned to Service → Controller.
7. Controller adds results to Model
8. Returns view "product-list"
9. Thymeleaf renders filtered result table Thymeleaf does th:each to loop results, display filtered products, and preserves filter values in form.
10. Browser displays advanced search result list

5.2: Category Filter

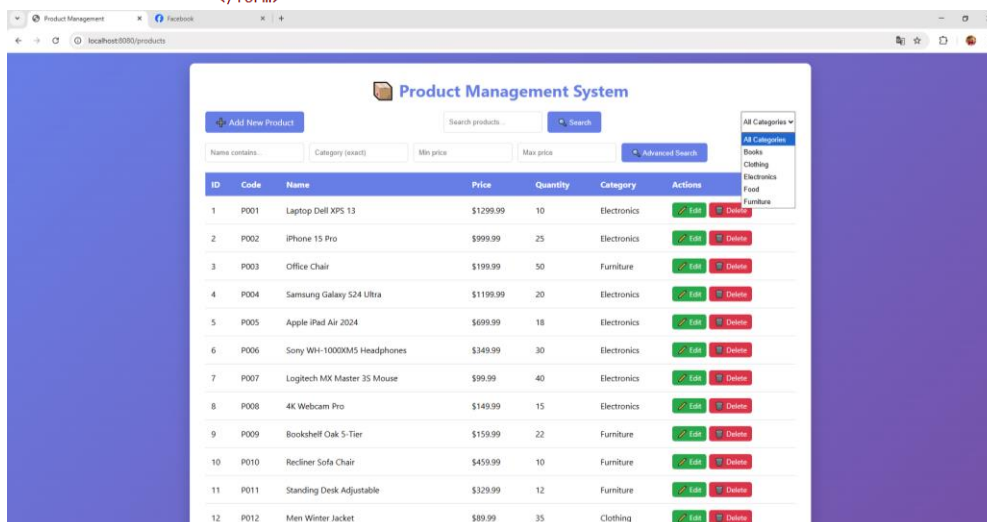
Add to ProductRepository:

```
@Query("SELECT DISTINCT p.category FROM Product p ORDER BY p.category")
List<String> findAllCategories();
```

Add filter dropdown to view:

```
<form th:action="@{/products}" method="get" class="search-form">
  <select name="category" onchange="this.form.submit()">
    <option value="">All Categories</option>

    <!-- Dynamic category list -->
    <option th:each="cat : ${categories}"
            th:value="${cat}"
            th:text="${cat}"
            th:selected="${cat == selectedCategory}">
    </option>
  </select>
</form>
```



A category filter with drop down menu

Explanation about the code flow of category filter

1. Browser → GET /products?category=Electronics
 - When user selects a category, browser sends: GET /products?category=Electronics
2. ProductController receives the request. The controller converts the query parameters including category selected by user.
3. Controller decides which service method to call.
 - Controller picks: products = productService.getProductsByCategory(category);.
4. Service (ProductServiceImpl.java) passes to Repository (ProductRepository.java).
5. Repository executes derived query method.
6. Database returns matching products List<Product>. These become List<Product> returned to Service → Controller.
7. Controller adds products + category info to Model.
8. View "product-list" is returned. Spring loads: src/main/resources/templates/product-list.html.
9. Thymeleaf marks selected category & displays list.
10. Filtered results displayed in browser

5.3: Search with Pagination

Modify repository method to use Pageable:

```
Page<Product> findByNameContaining(String keyword, Pageable pageable);
```

Update service interface and implementation

```
Page<Product> searchProducts(String keyword, Pageable pageable);

@Override
public Page<Product> searchProducts(String keyword, Pageable pageable) {
    return productRepository.findByNameContaining(keyword, pageable);
}
```

Update controller to handle pagination:

```
// Search products
@GetMapping("/search")
public String searchProducts(
    @RequestParam("keyword") String keyword,
    @RequestParam(defaultValue = "0") int page,
    @RequestParam(defaultValue = "5") int size,
    Model model) {

    Pageable pageable = PageRequest.of(page, size);
    Page<Product> productPage = productService.searchProducts(keyword, pageable);

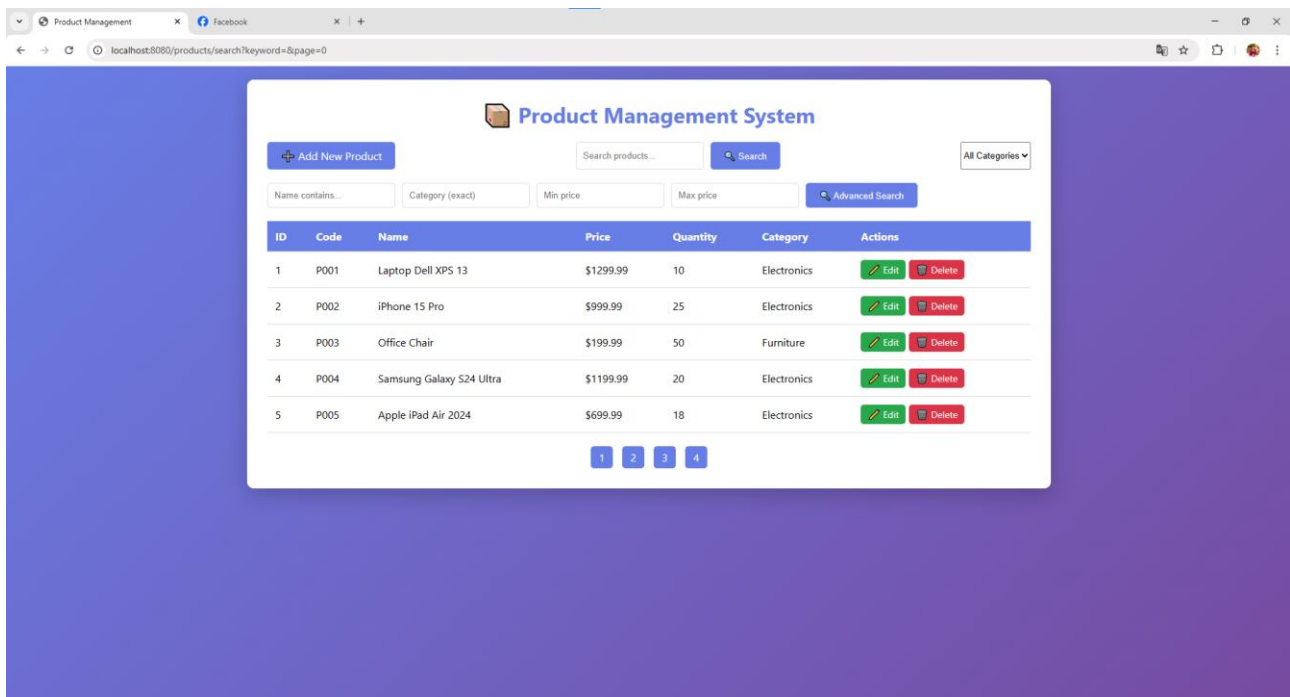
    model.addAttribute("products", productPage.getContent());
    model.addAttribute("currentPage", page);
    model.addAttribute("totalPages", productPage.getTotalPages());
    model.addAttribute("keyword", keyword);

    model.addAttribute("categories", productService.getAllCategories());

    return "product-list";
}
```

Update product-list.html to display pagination:

```
<div th:if="${totalPages > 1}" style="margin-top: 20px; text-align: center;">
    <span th:each="i : ${#numbers.sequence(0, totalPages - 1)}">
        <a th:href="@{/products/search(keyword=${keyword}, page=${i})}"
            th:text="${i + 1}"
            style="margin: 0 5px; padding: 6px 12px;
                border-radius: 5px; background: #667eea;
                color: white; text-decoration: none;
                display: inline-block;">
        </a>
    </span>
</div>
```



Searching with pagination

Explanation about the code flow of searching with pagination

1. Browser → GET /products/search?keyword=iphone&page=0&size=5
 - After user types iphone in the search field (for example iphone) and click search button, the browser sends the initial request with GET /products/search?keyword=iphone. The Spring sets keyword = “iphone”, page = 0, and size = 5.
2. The controller (ProductController.java file) receives the request.
3. Controller creates Pageable object. This creates the page number equals zero, page size equals 5.
4. Controller calls Service Layer
5. Service (ProductServiceImpl.java) calls Repository (ProductRepository.java).
6. JPA repository executes paginated query. Spring Data JPA turns findByNameContaining() into SQL for searching. It also executes an extra query to count total results for calculating the number of pages.
7. Repository returns a Page<Product> object. The object contains:
 - List<Product> → the products for the current page
 - totalPages → total number of pages
 - totalElements → total number of matched items
 - size → size per page
 - number → current page index
8. Controller adds data to Model. The model contains all data required to render product table, and render pagination buttons.
9. Controller returns “product-list”. The Thymeleaf will now render the list and pagination.
10. Thymeleaf displays paginated search results
11. Browser displays paginated results

EXERCISE 6: VALIDATION

6.1: Add Validation Annotations

Add to Product entity:

```

@NotBlank(message = "Product code is required")
@Size(min = 3, max = 20, message = "Product code must be 3-20 characters")
@Pattern(regexp = "^P\\d{3,}$", message = "Product code must start with P followed by numbers")
private String productCode;

@NotBlank(message = "Product name is required")
@Size(min = 3, max = 100, message = "Name must be 3-100 characters")
private String name;

@NotNull(message = "Price is required")
@DecimalMin(value = "0.01", message = "Price must be greater than 0")
@DecimalMax(value = "999999.99", message = "Price is too high")
private BigDecimal price;

@NotNull(message = "Quantity is required")
@Min(value = 0, message = "Quantity cannot be negative")
private Integer quantity;

@NotBlank(message = "Category is required")
private String category;

```

Update the pom.xml for the library validation

```

<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-validation</artifactId>
</dependency>

```

6.2: Add Validation in Controller

Update controller:

```

// Save product (create or update)
@PostMapping("/save")
public String saveProduct(
    @Valid @ModelAttribute("product") Product product,
    BindingResult result,
    Model model,
    RedirectAttributes redirectAttributes) {

    if (result.hasErrors()) {
        // Return form with validation errors
        return "product-form";
    }

    try {
        productService.saveProduct(product);
        redirectAttributes.addFlashAttribute("message",
            product.getId() == null ? "Product added successfully!" : "Product updated successfully!");
    } catch (Exception e) {
        redirectAttributes.addFlashAttribute("error", "Error: " + e.getMessage());
    }

    return "redirect:/products";
}

```

6.3: Display Validation Errors

Update product-form.html:

Product code:

```

<div class="form-group">
    <label for="productCode">Product Code *</label>
    <input type="text"
        id="productCode"
        th:field="*{productCode}"
        placeholder="Enter product code (e.g., P001)"
        th:errorclass="error" />
    <span th:if="${#fields.hasErrors('productCode')}"
        th:errors="*{productCode}"
        class="error-message"></span>
</div>

```

Product name:

```
<div class="form-group">
  <label for="name">Product Name *</label>
  <input type="text"
    id="name"
    th:field="*{name}"
    placeholder="Enter product name"
    th:errorclass="error" />
  <span th:if="${#fields.hasErrors('name')}}"
    th:errors="*{name}"
    class="error-message"></span>
</div>
```

Price:

```
<div class="form-group">
  <label for="price">Price ($) *</label>
  <input type="number"
    id="price"
    th:field="*{price}"
    step="0.01"
    min="0"
    placeholder="0.00"
    th:errorclass="error" />
  <span th:if="${#fields.hasErrors('price')}}"
    th:errors="*{price}"
    class="error-message"></span>
</div>
```

Quantity:

```
<div class="form-group">
  <label for="quantity">Quantity *</label>
  <input type="number"
    id="quantity"
    th:field="*{quantity}"
    min="0"
    placeholder="0"
    th:errorclass="error" />
  <span th:if="${#fields.hasErrors('quantity')}}"
    th:errors="*{quantity}"
    class="error-message"></span>
</div>
```

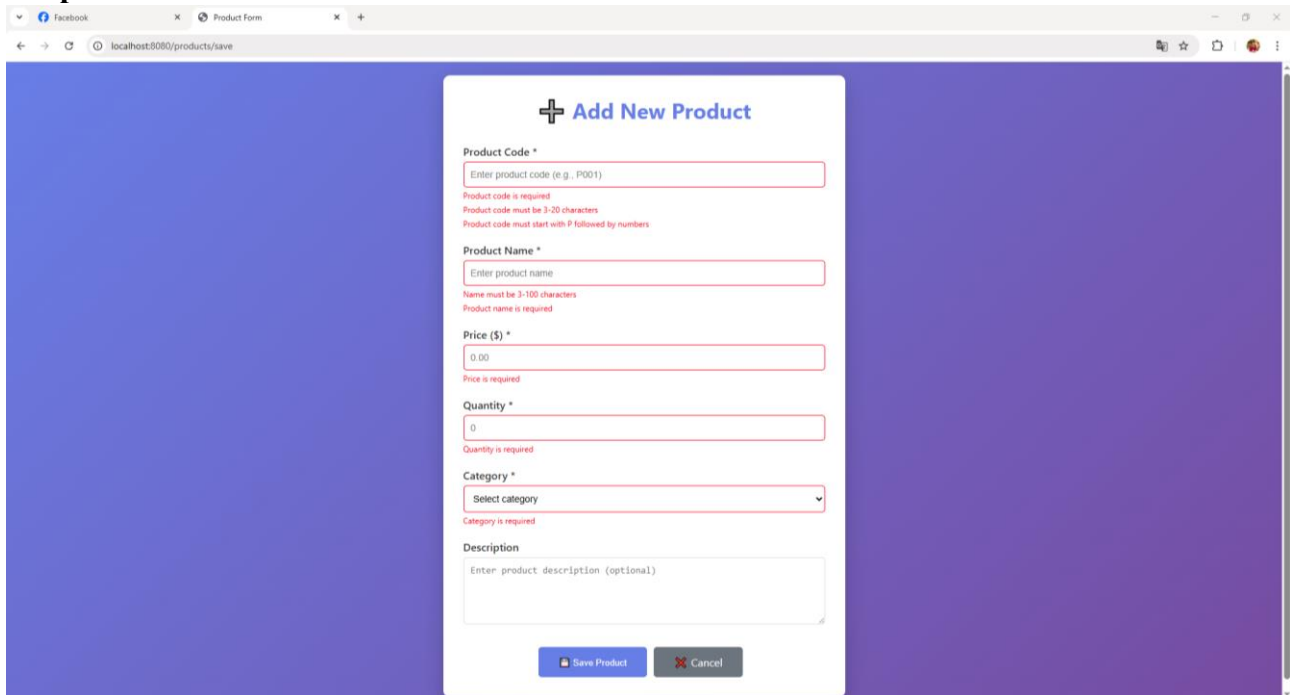
Category:

```
<div class="form-group">
  <label for="category">Category *</label>
  <select id="category" th:field="*{category}" th:errorclass="error">
    <option value="">Select category</option>
    <option value="Electronics">Electronics</option>
    <option value="Furniture">Furniture</option>
    <option value="Clothing">Clothing</option>
    <option value="Books">Books</option>
    <option value="Food">Food</option>
    <option value="Other">Other</option>
  </select>
  <span th:if="${#fields.hasErrors('category')}}"
    th:errors="*{category}"
    class="error-message"></span>
</div>
```

Add CSS for errors:

```
.error {  
  border-color: red !important;  
}  
  
.error-message {  
  color: red;  
  font-size: 12px;  
}
```

Output:



A screenshot of a web browser window showing a 'Product Form' at the URL 'localhost:8080/products/save'. The form is titled '+ Add New Product' and contains several input fields with red borders and error messages:

- Product Code ***: Input field with placeholder 'Enter product code (e.g., P001)'. Error messages: 'Product code is required', 'Product code must be 3-20 characters', 'Product code must start with P followed by numbers'.
- Product Name ***: Input field with placeholder 'Enter product name'. Error messages: 'Name must be 3-100 characters', 'Product name is required'.
- Price (\$) ***: Input field with placeholder '0.00'. Error message: 'Price is required'.
- Quantity ***: Input field with placeholder '0'. Error message: 'Quantity is required'.
- Category ***: Dropdown menu with placeholder 'Select category'. Error message: 'Category is required'.
- Description**: Textarea with placeholder 'Enter product description (optional)'.

At the bottom of the form are two buttons: 'Save Product' and 'Cancel'.

Display error message and show the correct form

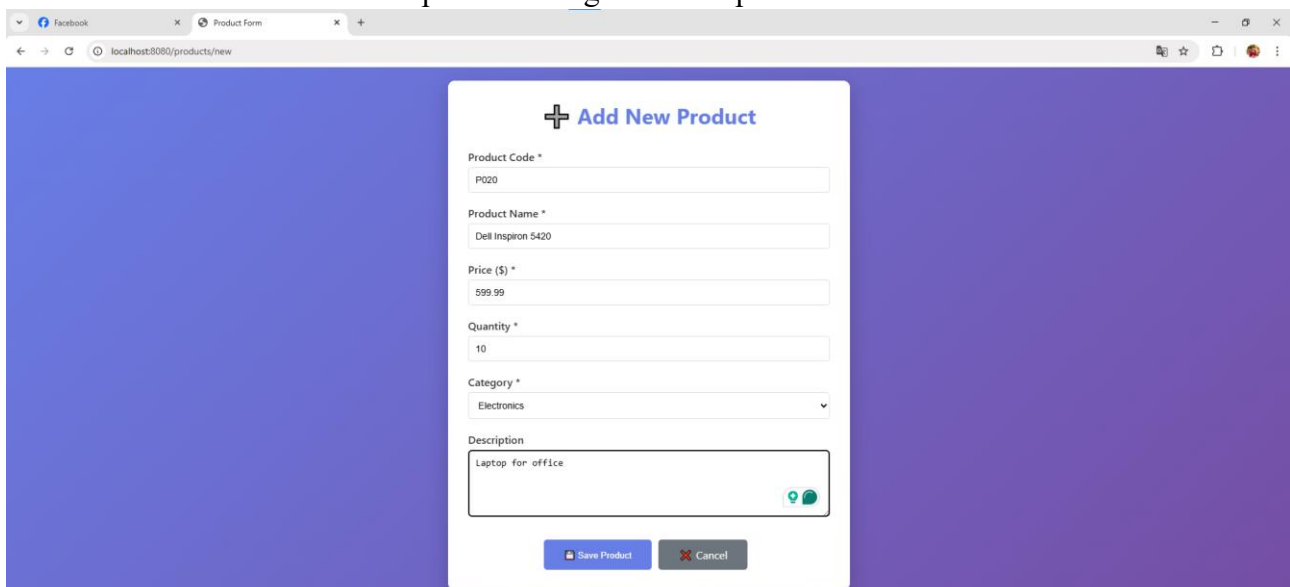
Product Code *



A close-up of the 'Product Code' input field. The field contains the text 'P01' and has a red border. Below the field, the error message 'Product code must start with P followed by numbers' is displayed in red text.

Product code must start with P followed by numbers

Input the wrong format of product code

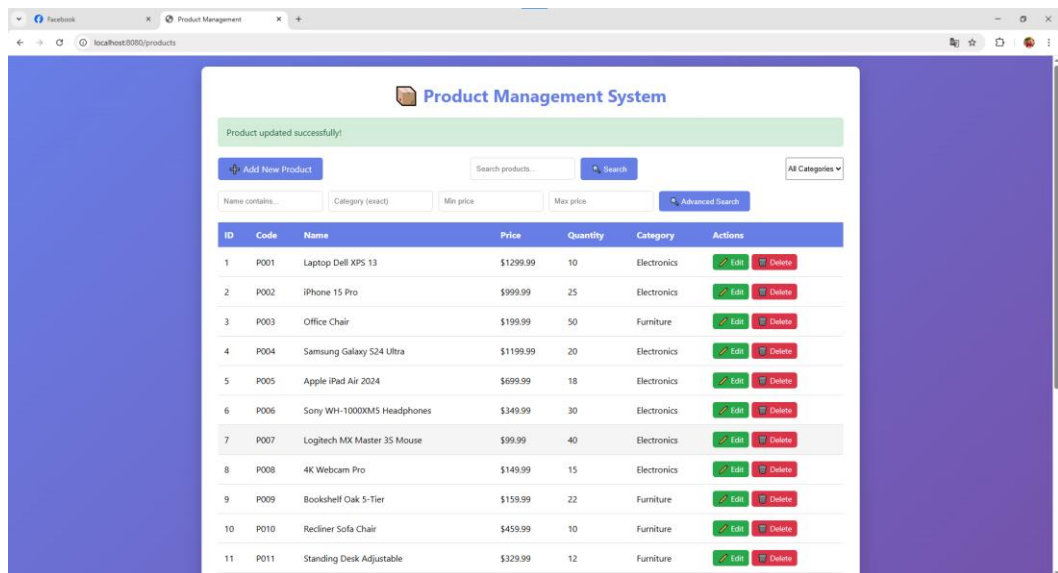


A screenshot of a web browser window showing the 'Product Form' at the URL 'localhost:8080/products/new'. The form is titled '+ Add New Product' and contains the following data:

- Product Code ***: Input field with value 'P020'.
- Product Name ***: Input field with value 'Dell Inspiron 5420'.
- Price (\$) ***: Input field with value '599.99'.
- Quantity ***: Input field with value '10'.
- Category ***: Dropdown menu with value 'Electronics'.
- Description**: Textarea with value 'Laptop for office'.

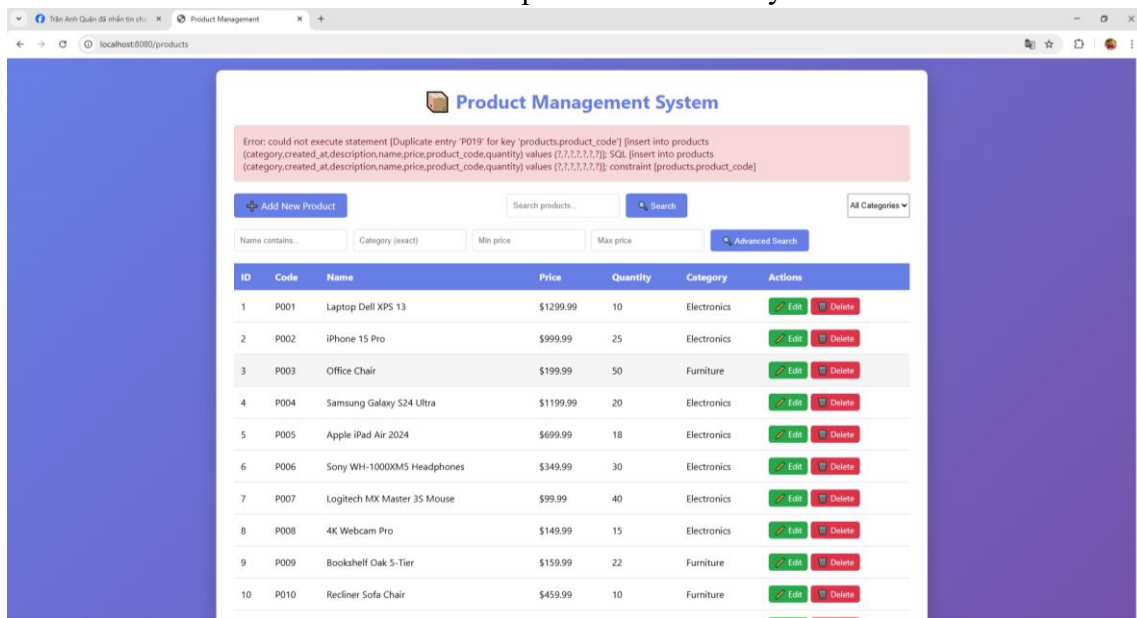
At the bottom of the form are two buttons: 'Save Product' and 'Cancel'.

Added new product form

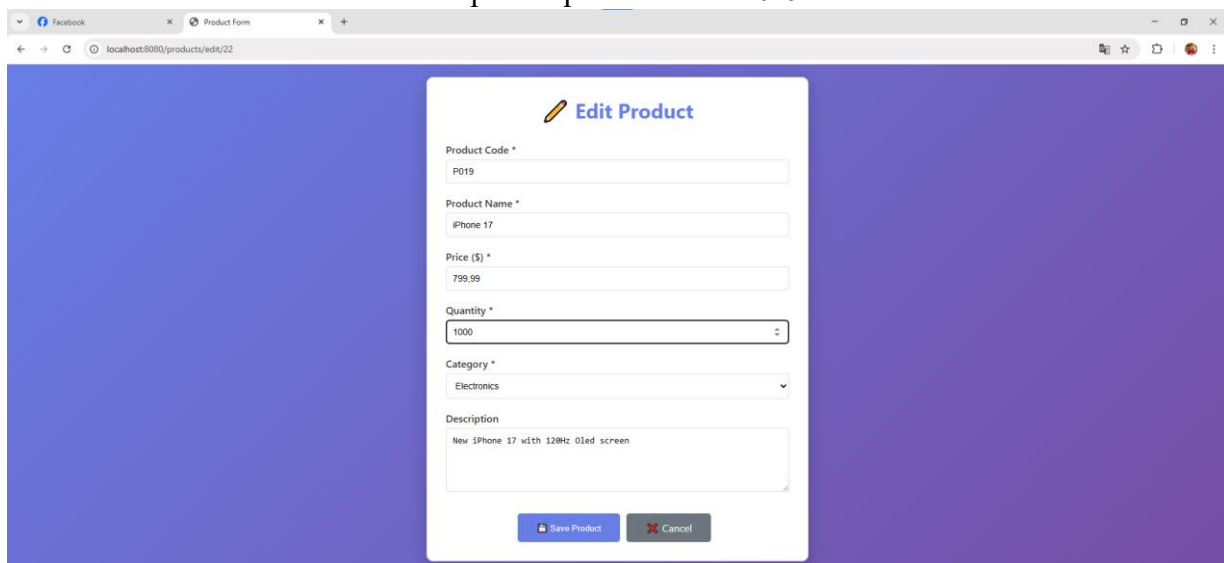


24 P020 Dell Inspiron 5420 \$599.99 10 Electronics [Edit](#) [Delete](#)

Added new product successfully



Duplicate product code P019



Edit the quantity of product from 100 to 1000

Product Management System

Product updated successfully!

[Add New Product](#) Search products: [Search](#) All Categories

Name contains: Category (exact): Min price: Max price: [Advanced Search](#)

ID	Code	Name	Price	Quantity	Category	Actions
1	P001	Laptop Dell XPS 13	\$1299.99	10	Electronics	Edit Delete
2	P002	iPhone 15 Pro	\$999.99	25	Electronics	Edit Delete
3	P003	Office Chair	\$199.99	50	Furniture	Edit Delete
4	P004	Samsung Galaxy S24 Ultra	\$1199.99	20	Electronics	Edit Delete
5	P005	Apple iPad Air 2024	\$699.99	18	Electronics	Edit Delete
6	P006	Sony WH-1000XM5 Headphones	\$349.99	30	Electronics	Edit Delete
7	P007	Logitech MX Master 3S Mouse	\$99.99	40	Electronics	Edit Delete
8	P008	4K Webcam Pro	\$149.99	15	Electronics	Edit Delete
9	P009	Bookshelf Oak 5-Tier	\$159.99	22	Furniture	Edit Delete
10	P010	Recliner Sofa Chair	\$459.99	10	Furniture	Edit Delete

22 P019 iPhone 17 \$799.99 1000 Electronics [Edit](#) [Delete](#)

Updated the quantity of product successfully

Explanation about the code flow of validation:

1. Browser → POST /products/save
 - After user fills in all information of product-form.html and clicks the save button, the browser sends a POST request containing form fields productCode, name, price, quantity, category, and description to controller.
2. The controller (ProductController.java file) receives the request. Spring performs validation before entering the method because of using @Valid.
3. Spring Validation Happens Automatically
 - Spring checks your Product entity's validation annotations.
4. Validation errors are stored in BindingResult. Spring populates the BindingResult: result.hasErrors() → true, The BindingResult object contains all field validation errors.
5. Controller checks validation
6. Controller returns back to product-form.html
 - If it fails the validation, the controller will return back to product-form.html and show the error message of failing the validation.
7. Thymeleaf displays validation errors
 - Thymeleaf extracts error messages from BindingResult and shows:
 - + "Name is required"
 - + "Price must be >= 0"
 - + "Quantity must be >= 0"
 - The user sees these printed directly below each field.
8. View is rerendered with error messages
9. If validation success, the controller calls the service layer for saving product. Service stores the product via repository.
10. Controller adds flash message that product added successfully
11. Redirect to product list. Browser now loads updated list.

EXERCISE 7: SORTING & FILTERING

7.1: Add Sorting

7.2: Filter by Category

7.3: Combined Sorting and Filtering

Update controller:

```
// List all products
@GetMapping
public String listProducts(
    @RequestParam(required = false) String sortBy,
    @RequestParam(defaultValue = "asc") String sortDir,
    @RequestParam(required = false) String category,
    Model model) {

    List<Product> products;

    Sort sort = null;
    if (sortBy != null) {
        sort = sortDir.equals(anObject: "asc")
            ? Sort.by(sortBy).ascending()
            : Sort.by(sortBy).descending();
    }

    if (category != null && !category.trim().isEmpty()) {
        if (sort != null) {
            products = productService.getProductsByCategory(category, sort);
        } else {
            products = productService.getProductsByCategory(category);
        }
    } else {
        if (sort != null) {
            products = productService.getAllProducts(sort);
        } else {
            products = productService.getAllProducts();
        }
    }

    model.addAttribute("products", products);
    model.addAttribute("sortBy", sortBy);
    model.addAttribute("sortDir", sortDir);
    model.addAttribute("selectedCategory", category);
    model.addAttribute("categories", productService.getAllCategories());

    return "product-list";
}
```

Update service to accept Sort parameter.

```
List<Product> getProductsByCategory(String category, Sort sort);

@Override
public List<Product> getProductsByCategory(String category, Sort sort) {
    return productRepository.findByCategory(category, sort);
}
```

Add sorting links to view:

```

<th>ID</th>
<th>Code</th>
<th th:classappend="${sortBy=='name'} ? 'active'">
  <a th:href="@{/products(sortBy='name', sortDir=${sortDir=='asc'? 'desc': 'asc'}, category=${selectedCategory})}">
    Name
    <span th:if="${sortBy=='name'}" th:text="${sortDir=='asc'? '↑': '↓'}"></span>
  </a>
</th>
<th th:classappend="${sortBy=='price'} ? 'active'">
  <a th:href="@{/products(sortBy='price', sortDir=${sortDir=='asc'? 'desc': 'asc'}, category=${selectedCategory})}">
    Price
    <span th:if="${sortBy=='price'}" th:text="${sortDir=='asc'? '↑': '↓'}"></span>
  </a>
</th>
<th th:classappend="${sortBy=='quantity'} ? 'active'">
  <a th:href="@{/products(sortBy='quantity', sortDir=${sortDir=='asc'? 'desc': 'asc'}, category=${selectedCategory})}">
    Quantity
    <span th:if="${sortBy=='quantity'}" th:text="${sortDir=='asc'? '↑': '↓'}"></span>
  </a>
</th>
<th th:classappend="${sortBy=='category'} ? 'active'">
  <a th:href="@{/products(sortBy='category', sortDir=${sortDir=='asc'? 'desc': 'asc'}, category=${selectedCategory})}">
    Category
    <span th:if="${sortBy=='category'}" th:text="${sortDir=='asc'? '↑': '↓'}"></span>
  </a>
</th>

```

Output:

The screenshot shows the 'Product Management System' interface. At the top, there's a search bar and a dropdown for 'All Categories'. Below that, there are filters for 'Name contains...', 'Category (exact)', 'Min price', and 'Max price'. The main table displays a list of products with columns: ID, Code, Name, Price, Quantity, Category, and Actions. The table is sorted by Name in descending order.

ID	Code	Name	Price	Quantity	Category	Actions
13	P013	Women Sports Leggings	\$29.99	60	Clothing	Edit Delete
14	P014	Unisex Hoodie Classic	\$39.99	55	Clothing	Edit Delete
11	P011	Standing Desk Adjustable	\$329.99	12	Furniture	Edit Delete
6	P006	Sony WH-1000XM5 Headphones	\$349.99	30	Electronics	Edit Delete
4	P004	Samsung Galaxy S24 Ultra	\$1199.99	20	Electronics	Edit Delete
10	P010	Recliner Sofa Chair	\$459.99	10	Furniture	Edit Delete
18	P018	Premium Green Tea 200g	\$12.99	90	Food	Edit Delete
17	P017	Organic Honey 500g	\$14.99	80	Food	Edit Delete
3	P003	Office Chair	\$199.99	50	Furniture	Edit Delete
12	P012	Men Winter Jacket	\$89.99	35	Clothing	Edit Delete

Filtering and sorting without filtering category

The screenshot shows the 'Product Management System' interface with the 'Category' dropdown set to 'Books'. The table displays a list of products filtered by the 'Books' category, sorted by Name in descending order.

ID	Code	Name	Price	Quantity	Category	Actions
16	P016	Clean Code	\$49.99	28	Books	Edit Delete
15	P015	Machine Learning with Python	\$54.99	25	Books	Edit Delete

Filtering and sorting with category = "Books"

Explanation about the code flow of filtering and sorting

1. Browser → GET /products
 - After user choosing sorting link and category via drop down menu (for example sorted by price with Electronics category, the browser sends a GET method to the controller such as: GET /products?category=Electronics&sortBy=price&sortDir=asc.
2. Productcontroller receives request. Spring injects the sortBy, sortDir, and category.
3. Controller builds sorting object
4. Controller chooses which service method to call based on the user choice.
 - If the user selects a category with sorting via the title of table, the method `getProductsByCategory(category, sort)` will be executed.
 - Otherwise, the method `getProductsByCategory(category)` will be executed.
5. Service layer (`ProductServiceImpl.java` file) executes filter + sort
6. Repository executes database query
7. Repository returns sorted + filtered list
8. Controller adds model attributes such as products, sortBy, sortDir, category. These are required for showing sorting arrows, preserving selected category, building link correctly, repopulating dropdown menu.
9. Controller returns view
10. Thymeleaf renders sorted + filtered data

EXERCISE 8: STATISTICS DASHBOARD

8.1: Add Statistics Methods

Add to ProductRepository:

```
@Query("SELECT COUNT(p) FROM Product p WHERE p.category = :category")
long countByCategory(@Param("category") String category);

@Query("SELECT SUM(p.price * p.quantity) FROM Product p")
BigDecimal calculateTotalValue();

@Query("SELECT AVG(p.price) FROM Product p")
BigDecimal calculateAveragePrice();

@Query("SELECT p FROM Product p WHERE p.quantity < :threshold")
List<Product> findLowStockProducts(@Param("threshold") int threshold);

@Query(value = "SELECT * FROM products ORDER BY id DESC LIMIT 5", nativeQuery = true)
List<Product> findRecentProducts();
```

8.2: Create Dashboard Controller

```
@Controller
@RequestMapping("/dashboard")
public class DashboardController {

    @Autowired
    private ProductService productService;

    @GetMapping
    public String showDashboard(Model model) {

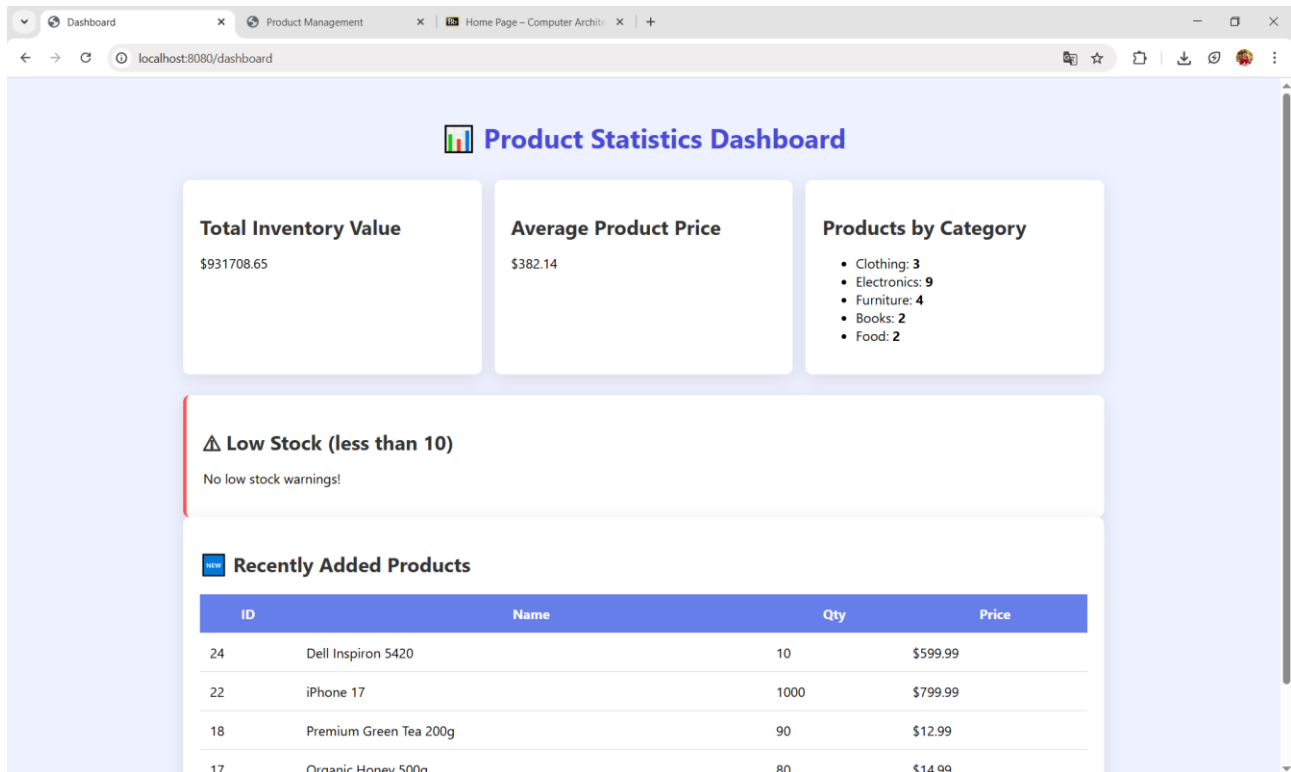
        List<String> categories = productService.getAllCategories();

        // Map for category statistics
        Map<String, Long> categoryStats = new HashMap<>();
        for (String c : categories) {
            categoryStats.put(c, productService.countProductsByCategory(c));
        }

        model.addAttribute("categoryStats", categoryStats);
        model.addAttribute("totalValue", productService.getTotalValue());
        model.addAttribute("avgPrice", productService.getAveragePrice());
        model.addAttribute("lowStock", productService.getLowStockProducts(threshold: 10));
        model.addAttribute("recentProducts", productService.getRecentProducts());

        return "dashboard";
    }
}
```

8.3: Create Dashboard View



Explanation about the code flow of displaying dashboard:

1. Browser → GET /dashboard. When the user types /dashboard in the browser address bar, there is a GET method requests to the controller (DashboardController.java file).
2. Spring boot finds dashboardcontroller (@GetMapping). Spring sees @GetMapping and calls this method for /dashboard.
3. Controller calls service to get statistics:
 - Count all products
 - Total inventory value
 - Average price
 - Load all categories
 - Count products per category
 - Low-stock alert
 - Recent products
4. Service returns results → controller receives data
5. Controller adds data to model. The Model is a storage container that Thymeleaf reads when rendering HTML.
6. Controller returns view name
7. Thymeleaf renders html. Inside dashboard.html, there are the views displayed:
 - Total products
 - Total inventory value
 - Average price
 - Category counts
 - Low-stock products
 - Recent products
8. Spring sends final html back to browser

BONUS 1: REST API Endpoints

Create RestController:

```

// GET all products
@GetMapping
public ResponseEntity<List<Product>> getAllProducts() {
    List<Product> list = productService.getAllProducts();
    return ResponseEntity.ok(list); // 200 OK + JSON array
}

// GET product by ID
@GetMapping("/{id}")
public ResponseEntity<Product> getProduct(@PathVariable Long id) {
    Optional<Product> product = productService.getProductById(id);

    return product
        .map(ResponseEntity::ok) // 200 OK
        .orElseGet(() -> ResponseEntity.notFound().build()); // 404 NOT FOUND
}

// CREATE new product
@PostMapping
public ResponseEntity<Product> createProduct(@RequestBody Product product) {
    try {
        Product savedProduct = productService.saveProduct(product);

        return ResponseEntity.created(
            URI.create("/api/products/" + savedProduct.getId())
        ).body(savedProduct); // 201 CREATED + JSON body
    } catch (Exception e) {
        return ResponseEntity.badRequest().build(); // 400 BAD REQUEST
    }
}

// UPDATE existing product
@PutMapping("/{id}")
public ResponseEntity<Product> updateProduct(
    @PathVariable Long id,
    @RequestBody Product productData) {

    Optional<Product> existing = productService.getProductById(id);
    if (existing.isEmpty()) {
        return ResponseEntity.notFound().build(); // 404
    }

    Product product = existing.get();

    // Update fields
    product.setProductCode(productData.getProductCode());
    product.setName(productData.getName());
    product.setPrice(productData.getPrice());
    product.setQuantity(productData.getQuantity());
    product.setCategory(productData.getCategory());
    product.setDescription(productData.getDescription());

    Product updatedProduct = productService.saveProduct(product);

    return ResponseEntity.ok(updatedProduct); // 200 OK
}

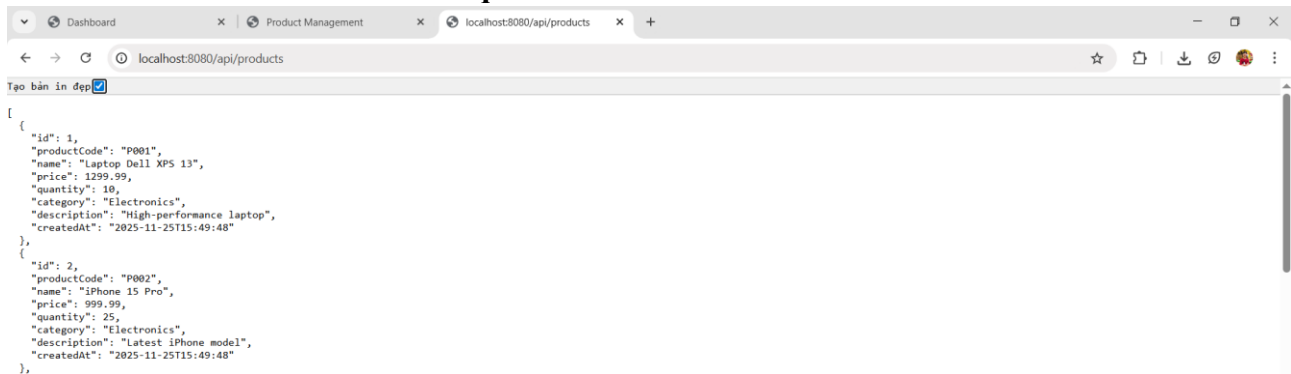
// DELETE product
@DeleteMapping("/{id}")
public ResponseEntity<Void> deleteProduct(@PathVariable Long id) {
    Optional<Product> product = productService.getProductById(id);

    if (product.isEmpty()) {
        return ResponseEntity.notFound().build(); // 404
    }

    productService.deleteProduct(id);
    return ResponseEntity.noContent().build(); // 204 NO CONTENT
}

```

Test with web browser for GET requests:



Explanation about the code flow of REST API Endpoints:

1. Browser / Client → API Request
2. Request arrives at ProductRestController
 - @RestController = automatically converts Java objects → JSON
 - @RequestMapping("/api/products") = every endpoint inside starts with: /api/products/...
3. Controller handles GET /api/products
 - Client sends request
 - Spring finds @GetMapping
 - Calls productService.getAllProducts()
 - Service → Repository → findAll()
 - Repository queries: SELECT * FROM product;.
 - Returns List<Product>
 - Spring converts list → JSON
 - Response sent back to client
4. Controller handles GET /api/products/{id}
 - Client sends e.g.: GET /api/products/5
 - @PathVariable extracts id = 5
 - Service calls repository: SELECT * FROM product WHERE id = 5;.
 - If found → return **200 OK** + JSON
 - If not found → return **404 Not Found**
5. Controller handles POST /api/products
 - Client sends JSON body with data
 - @RequestBody converts JSON → Product object
 - Service saves product
 - Repository executes: INSERT INTO product (...) VALUES (...);.
 - Database generates ID
 - Controller returns 201 Created
6. Controller handles PUT /api/products/{id}
 - Client sends: PUT /api/products/10 with JSON body with update data.
 - Spring maps JSON → Product object
 - Service checks if ID exists
 - If exists: UPDATE product SET ... WHERE id = 10;.. If not exists → return 404 Not Found.
 - Return 200 OK + updated JSON
7. Controller handles DELETE /api/products/{id}
 - Client sends: DELETE /api/products/10
 - Service checks if product exists. If not exists → 404 Not Found

- Repository runs: DELETE FROM product WHERE id = 10;.
- Controller returns 204 No Content.

BONUS 3: Export to Excel

Add dependency:

```
<dependency>
  <groupId>org.apache.poi</groupId>
  <artifactId>poi-ooxml</artifactId>
  <version>5.2.3</version>
</dependency>
```

Create ExportController:

```
@GetMapping("/excel")
public void exportToExcel(HttpServletResponse response) throws IOException {

    // Set response (force download)
    response.setContentType("application/octet-stream");
    response.setHeader("Content-Disposition", "attachment; filename=products.xlsx");

    // Get product list
    List<Product> products = productService.getAllProducts();

    // Create workbook
    Workbook workbook = new XSSFWorkbook();
    Sheet sheet = workbook.createSheet("Products");

    // Header style
    CellStyle headerStyle = workbook.createCellStyle();
    Font font = workbook.createFont();
    font.setBold(true);
    headerStyle.setFont(font);

    // Create header row
    Row header = sheet.createRow(0);

    String[] columns = {"ID", "Product Code", "Name", "Price", "Quantity", "Category", "Created At"};

    for (int i = 0; i < columns.length; i++) {
        Cell cell = header.createCell(i);
        cell.setCellValue(columns[i]);
        cell.setCellStyle(headerStyle);
    }

    // Write product data
    int rowNum = 1;
    for (Product p : products) {
        Row row = sheet.createRow(rowNum++);

        row.createCell(0).setCellValue(p.getId());
        row.createCell(1).setCellValue(p.getProductCode());
        row.createCell(2).setCellValue(p.getName());

        // Price (BigDecimal to double)
        BigDecimal price = p.getPrice();
        row.createCell(3).setCellValue(price != null ? price.doubleValue() : 0);

        row.createCell(4).setCellValue(p.getQuantity());
        row.createCell(5).setCellValue(p.getCategory());

        if (p.getCreatedAt() != null) {
            row.createCell(6).setCellValue(p.getCreatedAt().toString());
        } else {
            row.createCell(6).setCellValue("N/A");
        }
    }

    // Auto-size columns
    for (int i = 0; i < columns.length; i++) {
        sheet.autoSizeColumn(i);
    }

    // Write to browser
    workbook.write(response.getOutputStream());
    workbook.close();
}
```

Explanation about the code flow of export to Excel:

1. Browser → User clicks "Export to Excel" button. When clicked, the browser sends: GET /export/excel.
2. Request arrives at ExportController
 - Spring finds the matching @GetMapping("/excel").
 - Calls the exportToExcel() method.
 - No HTML view is returned — instead we directly write to HttpServletResponse.
3. Controller sets response headers. This tells the browser that the user is downloading an Excel file. Without this, the browser will not download the file.
4. Controller fetches data from the database
 - The controller calls the Service/Repository layer.
 - JPA executes: SELECT * FROM product;.
 - Return List<Product>.
5. Controller creates Excel workbook (Apache POI)
 - A new Excel file is created in memory.
 - A sheet named "Products" is added.
6. Create header row (first row with column names)
 - Sheet row 0 is used as the header.
 - Column titles are written using setCellValue.
7. Loop through product list and write rows
 - Starts at row 1 (below header).
 - Fills each cell based on product data.
 - Converts price to double for Excel.
8. Write Excel file to HTTP response
 - Excel workbook content is written directly on the browser.
 - File is NOT stored on the server.
 - Browser immediately downloads it.